

PPT1 封面页

大家好。

我们小组汇报的题目是：

《面向AI生成代码的人工审核与智能支持现状——一项经验软件工程案例研究》

随着ChatGPT、GitHub Copilot以及各种代码大模型被广泛应用，越来越多的软件代码已经不是由程序员逐行编写，而是由AI辅助生成。

那么问题来了：

当AI开始写代码之后，代码评审是否会变得更容易？智能评审工具又能否真正帮助开发者发现问题？

围绕这些问题，我们开展了本次案例研究。

PPT2 研究背景

首先介绍研究背景。

近年来，大语言模型正在快速进入软件开发流程。

例如GitHub Copilot、ChatGPT以及DeepSeek-Coder等工具，已经能够生成完整函数甚至模块级代码。

这种变化极大提高了开发效率。

但与此同时，也给软件质量保障带来了新的挑战。

因为AI能够快速生成代码，并不意味着生成的代码一定正确。

因此代码评审依然是保证软件质量的重要环节。

本研究正是在这样的背景下展开。

PPT3 研究动机与挑战

AI生成代码给代码评审带来了三个主要挑战。

第一个挑战是代码规模激增。

过去开发者编写代码需要较长时间，而AI几秒钟就能生成大量代码，导致人工评审压力显著增加。

第二个挑战是缺陷模式发生变化。

AI代码通常语法正确、格式规范，但可能隐藏逻辑漏洞、安全缺陷或者上下文不一致问题。

这些问题比传统语法错误更难发现。

第三个挑战是信任问题。

评审者可能过度相信AI建议，也可能完全不相信AI结果。

因此如何实现高效的人机协同成为新的研究问题。

PPT4 研究问题设计

基于上述背景，我们提出了一个总体研究问题：

面向AI生成代码的人工审核与智能支持现状是什么？

进一步细化为四个研究问题。

RQ1：

AI生成代码是否会增加人工评审负担？

RQ2：

智能评审工具能否有效提升评审质量？

RQ3：

不同技术路线的评审工具之间是否存在显著差异？

RQ4：

代码复杂度等因素是否会影响工具效果？

接下来所有实验设计都围绕这四个问题展开。

PPT5 案例研究设计

接下来介绍案例研究设计。

我们的案例对象是AI生成代码的人机协同评审过程。

研究目标是分析AI代码带来的评审负担，并评估不同智能支持技术的实际效果。

研究共包含三个典型案例。

分析单元为单个代码片段及其对应评审行为。

数据来源包括：

行为日志、

评审结果、

问卷调查以及访谈数据。

在分析方法方面，我们结合使用了ANOVA方差分析、回归分析以及扎根理论等定量和定性研究方法。

力求从多个角度分析问题。

PPT6 技术路线设计

为了比较不同智能化程度的评审工具，我们设计了三种技术路线。

第一种是Method-A。

采用SonarQube静态分析工具。

主要依赖预定义规则进行检测。

第二种是Method-B。

采用机器学习方法。

通过代码复杂度、嵌套深度等指标训练XGBoost模型，对缺陷风险进行预测和排序。

第三种是Method-C。

采用基于CodeLlama的大语言模型评审Agent。

结合LoRA微调和RAG检索增强机制。

不仅能够发现问题，还能够解释原因并生成修复建议。

这三种方法代表了从规则驱动到生成式AI的技术演进路径。

PPT7 数据集构建

为了尽可能贴近真实开发场景，我们采用了双源混合数据集。

一部分来自GitHub开源项目。

共抽取100个真实Pull Request样本。

另一部分来自AI生成代码。

我们设计了10类典型开发任务，例如线程同步、权限验证、缓存系统等。

随后注入五类典型缺陷。

包括：

资源泄漏、

权限绕过、

竞态条件、

边界溢出以及不安全反序列化。

最终形成212个有效代码单元。

同时覆盖Java和Python两种语言。

PPT8 实验过程

接下来介绍实验实施过程。

我们从南京大学软件学院招募参与者。

经过预实验筛选后，共有24名参与者进入正式实验。

随后随机分配到三个实验组。

每组8人。

为了避免学习效应和顺序偏差，我们采用拉丁方设计控制样本顺序。

此外还设计了双阶段评审机制。

参与者首先独立完成代码评审。

然后再查看智能工具建议。

最后提交修改后的结果。

这样能够有效避免直接依赖AI建议产生锚定效应。

PPT9 评价指标体系

为了全面评价工具效果，我们构建了三类指标体系。

第一类是质量指标。

包括Recall、Precision和F1-Score。

第二类是效率指标。

包括审核时长和定位延迟。

第三类是主观体验指标。

包括NASA-TLX认知负荷量表以及工具信任度问卷。

通过这三个维度，可以同时评价工具是否准确、高效以及是否真正被用户接受。

PPT10 RQ1结果

首先回答RQ1。

AI生成代码是否增加了评审负担？

结果显示：

AI代码的NASA-TLX平均得分达到48.9。

而人工代码仅为36.4。

差异具有统计显著性。

同时审核时间也从38秒增加到53秒左右。

增长接近38%。

访谈结果进一步发现：

很多参与者认为AI代码表面上格式规范、命名合理，但实际上隐藏着较深层次的逻辑问题。

容易产生“代码已经正确”的错觉。

因此我们认为：

AI生成代码确实显著增加了人工评审负担。

PPT11 RQ2结果

接下来回答RQ2。

智能评审工具是否有效？

从F1-Score结果可以看到。

Control组为0.65。

Method-A提升到0.69。

Method-B提升到0.76。

Method-C进一步提升到0.86。

方差分析结果显示差异显著。

其中Method-C效果最为突出。

说明智能支持工具确实能够提高评审质量。

尤其是基于大语言模型的评审Agent表现最佳。

PPT12 RQ3结果

第三个问题是：

不同技术路线是否存在差异？

结果表明：

Method-C在Precision、Recall以及效率指标上均表现最好。

审核时间从52秒下降到31秒。

降低超过40%。

同时用户信任度达到4.4分。

显著高于其他方法。

访谈中超过70%的参与者认为Method-C提供的解释更加清晰，更符合真实评审过程。

因此不同技术路线之间确实存在明显差异。

LLM评审Agent表现最优。

PPT13 RQ4结果

接下来回答RQ4。

代码复杂度是否会影响工具效果？

回归分析显示：

随着代码复杂度提高，人类评审质量持续下降。

传统静态分析工具效果也有所退化。

但Method-C反而表现出更大的优势。

也就是说：

任务越复杂，大语言模型带来的收益越明显。

因此我们提出一个新的现象：

Difficult-Task Amplifier。

即高难度任务放大器效应。

说明LLM在复杂代码场景下具有更高价值。

PPT14 核心发现

结合量化分析与访谈结果。

我们进一步提出了一个人机协同机制模型。

即：

认知卸载与决策增强双路径模型。

首先。

智能体能够提前完成控制流分析、漏洞定位和异常传播分析。

帮助用户减少推理负担。

这属于认知卸载。

另一方面。

智能体还能生成修复建议和重构方案。

帮助用户更快完成决策。

这属于决策增强。

最终使定位延迟下降超过60%。

不过值得注意的是。

即使在Method-C条件下，参与者仍然修改了约23%的AI建议。

说明AI并没有取代人类。

人类依然是最终决策者。

PPT15 总结与启示

最后总结本研究的主要结论。

第一，

AI生成代码显著增加了人工评审负担。

第二，

智能评审工具能够有效提高评审质量。

第三，

LLM评审Agent在质量、效率和体验方面表现最佳。

第四，

代码越复杂，LLM带来的收益越明显。

第五，

当前最优模式仍然是Human-in-the-Loop的人机协同评审。

对于实践而言，我们建议未来代码评审工具采用双阶段设计模式。

先由开发者独立评审，再引入AI辅助。

从而充分发挥人类判断能力与AI分析能力的优势。

我的汇报到这里结束。

感谢各位老师和同学的聆听，欢迎大家提出问题。谢谢大家。🎉